HCLTech Career Shaper Programme

IV Year Industry Project | Academic Year 2026–27

PROJECT REPORT

Time-Series Anomaly Detection in Financial Transactions

Project No. 58

Project Information	Details
Domain	Data Science / Python
Difficulty	Advanced
Duration	4 Weeks
Format	Individual Project (1 Learner)
Learner Name	Shivam Verma
College / Campus	ABES Engineering College, Ghaziabad
Roll Number	2300320130234
Mentor Name	Natesan Rajan
Submission Date	29 October 2026

Abstract

Financial transaction systems generate records across accounts, amounts, transaction types, merchant categories, and time. Unusual records can be useful signals for review, but a detector must be implemented carefully because incorrect alerts may affect account access. This project implements a working unsupervised baseline for time-series financial transaction anomaly detection. The prototype uses Python, Pandas, Scikit-learn's Isolation Forest, and a Flask web interface. The application loads the supplied CSV, validates and engineers features, runs detection with a configurable contamination parameter, displays predicted flags and model scores, and offers a downloadable results CSV.

The supplied dataset contains 500 rows and 7 columns, with 3 positive values in the existing is_anomaly field (0.6%). The source and meaning of the existing anomaly labels/scores are not verified. The model therefore excludes is_anomaly and anomaly_score from model inputs, and the results are treated as exploratory predictions rather than validated fraud decisions. Automated verification reported 24 passing tests and successful checks of the dashboard homepage, detection endpoint, and download endpoint. The project does not establish the brief's false-positive-rate target of below 0.1%, and it is not a production financial-risk system.

Keywords

Anomaly detection, financial transactions, time series, Isolation Forest, feature engineering, Flask, unsupervised learning.

Report Scope and Evidence

This report distinguishes implemented functionality from suggested approaches in the brief. Isolation Forest is implemented; Autoencoder and LSTM-AE are not implemented in the current prototype. The report does not invent a mentor approval, legal certification, production deployment, or performance score.

Contents

1. 1. Introduction and Business Objective
2. 2. Problem Statement and Project Objectives
3. 3. Dataset Description and Data Quality
4. 4. Exploratory Data Analysis
5. 5. System Design and Architecture
6. 6. Methodology and Feature Engineering
7. 7. Application Implementation
8. 8. Testing, Evaluation and Results
9. 9. Ethical, Privacy and Operational Considerations
10. 10. Limitations and Future Work
11. 11. Conclusion
12. References
13. Appendix A: Software Verification Summary
14. Appendix B: Learner Completion Checklist

1. Introduction and Business Objective

Financial transaction monitoring is a practical application of data science. Transaction streams can contain ordinary variation caused by account behaviour, merchant type, transaction amount, and timing. Some records may differ substantially from the general pattern and deserve additional review. Anomaly detection can help prioritise such records, but an anomaly flag is not proof of fraud and should not automatically trigger account restrictions.

The official Project 58 brief asks for a working time-series anomaly detection solution, EDA with at least three visualisations, a cleaned/pre-processed dataset, a complete data dictionary, measurable validation, a structured individual report of at least eight pages, a short demo or live demonstration, and a signed ethical declaration.

1.1 Business Objective

The objective of this implementation is to demonstrate a working application that analyses transaction records and highlights potentially unusual records through an unsupervised baseline. The user can inspect the dataset summary, select a contamination value, execute Isolation Forest, review the predicted flags and scores, and download the results for further examination.

1.2 Intended Users

- Data analysts who need an initial review queue for unusual transaction records.
- Project evaluators who need to inspect a reproducible end-to-end machine-learning prototype.
- Developers who may extend the baseline after obtaining better-labelled and larger datasets.

1.3 Expected Value

The prototype demonstrates the connection between data loading, feature preparation, model execution, a web interface, and exportable outputs. Its business value is currently demonstrative: it shows how a review workflow could be organised, but no reduction in fraud loss, false-positive rate, or operational cost has been measured.

2. Problem Statement and Project Objectives

2.1 Problem Statement

Financial service teams may need to review large numbers of transactions to identify unusual activity, while the available data may have few reliable anomaly labels. Manual review alone can be slow, and simplistic thresholds may overlook unusual combinations of transaction amount, time, type, and merchant category. This project builds an unsupervised baseline and a local dashboard to help identify records for human review without treating a model flag as a confirmed fraud finding.

2.2 Objectives

- Load and validate the provided dataset while preserving the original source file.
- Explore transaction amount, time distribution, transaction types, merchant categories, and the existing label distribution.
- Engineer model features from timestamps, transaction amounts, transaction type, and merchant category.
- Exclude the existing is_anomaly and anomaly_score fields from model inputs to reduce label leakage risk; exclude account_id and raw timestamp as direct model inputs.
- Implement a configurable Isolation Forest baseline with reproducible random state.
- Provide a Flask dashboard for dataset overview, detection execution, result review, and CSV download.
- Run automated tests and verify core web endpoints and input validation.
- Document the limitations of the data and avoid unsupported claims about FPR or production reliability.

2.3 Scope Boundaries

The current implementation uses Isolation Forest only. Deep-learning models listed in the project description (Autoencoder and LSTM-AE) are not part of the verified implementation. No production database, account-blocking workflow, live payment integration, or automated appeal portal is implemented.

3. Dataset Description and Data Quality

The project uses the supplied dataset.csv file. Its columns correspond to the seven fields named in the official brief. The file was read and inspected for shape, missing values, duplicate rows, value distributions, timestamp range, and label counts.

Field	Observed value
Dataset file	dataset.csv
Rows × columns	500 × 7
Missing values	0
Duplicate rows	0
Unique account IDs	468
Timestamp range	2024-01-01 00:26:00 to 2024-01-06 09:10:00
Transaction amount range	6.07 to 1471.92
Mean / median amount	152.04 / 92.62
Existing is_anomaly labels	497 zeros, 3 ones (0.6% positive)
Existing anomaly_score range	0.0043 to 0.9978; mean 0.5187

3.1 Data Dictionary

Column	Meaning in this project	Use in baseline
timestamp	Date/time of transaction	Parsed to derive time features; raw timestamp excluded
account_id	Anonymised account identifier	Excluded from model features
trans_amount	Transaction amount	Numeric model feature
transaction_type	Credit, Debit, or Transfer	One-hot encoded categorical feature
merchant_category	Merchant group such as Travel or Grocery	One-hot encoded categorical feature
anomaly_score	Pre-existing score; provenance unknown	Excluded from model inputs
is_anomaly	Existing binary label; provenance unknown	Excluded from model inputs; exploratory reference only

Important data-quality note: the uploaded data_dictionary.csv initially contained placeholder descriptions. The report explains each field here, but the data dictionary file itself should be checked and updated in the project folder before submission if its description column still contains placeholders.

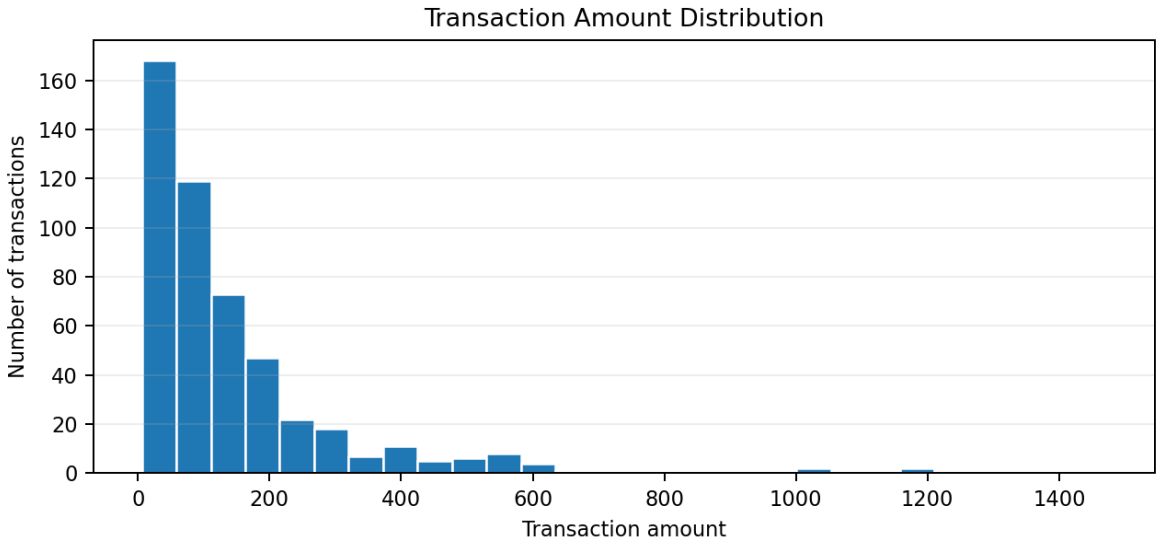
3.2 Data Integrity

The uploaded dataset examined for this report has SHA-256 49a5e6bc678abdc105146bdbeb8a45496aa12470b639bd29779ce6ab1d8ebfc5. This is an integrity fingerprint for this copy, not a claim that the dataset is licensed or independently authenticated. Previous local project verification reported that the original dataset file remained unchanged during model and dashboard work.

4. Exploratory Data Analysis

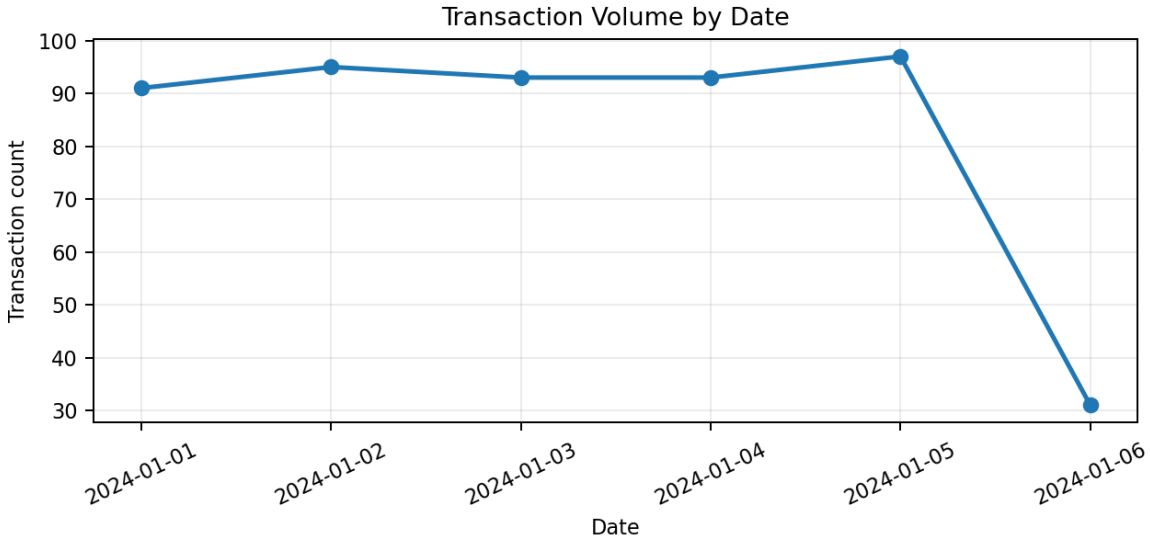
EDA was used to understand the available transaction records before modelling. Four visualisations are included, exceeding the brief's minimum of three visualisations.

4.1 Transaction Amount Distribution



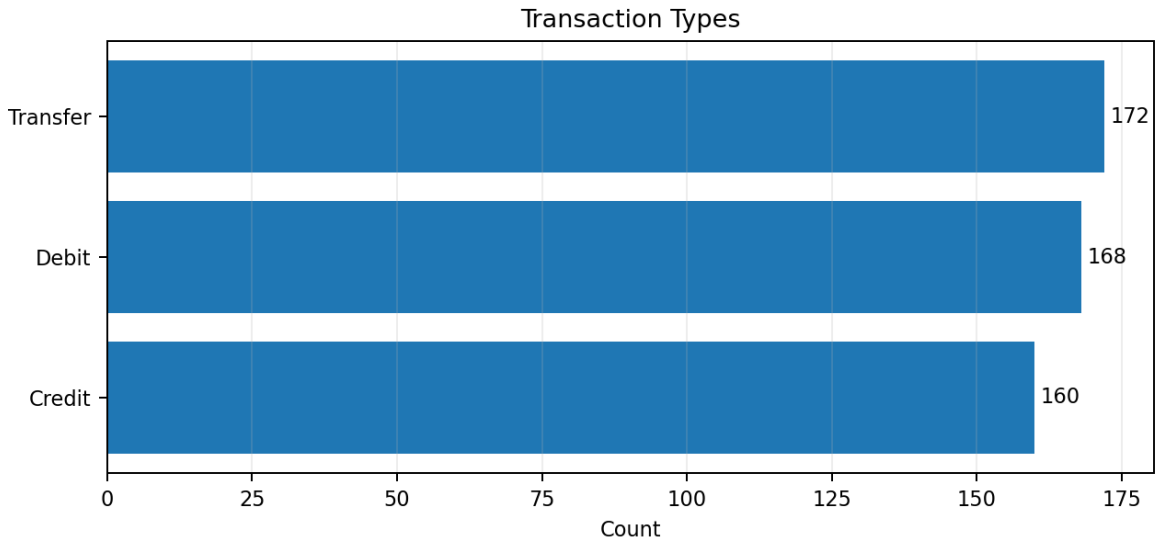
Transaction amounts range from 6.07 to 1471.92; the mean is 152.04 and the median is 92.62. The difference between the mean and median suggests that the distribution is right-skewed, with some high-value transactions. Amount alone is not enough to classify a transaction as fraudulent.

4.2 Transaction Volume Over Time



The records span 2024-01-01 through 2024-01-06. The dataset is time-ordered and covers only a short interval, so it is useful for demonstrating time-derived features but not for establishing long-term seasonality.

4.3 Transaction Type Distribution

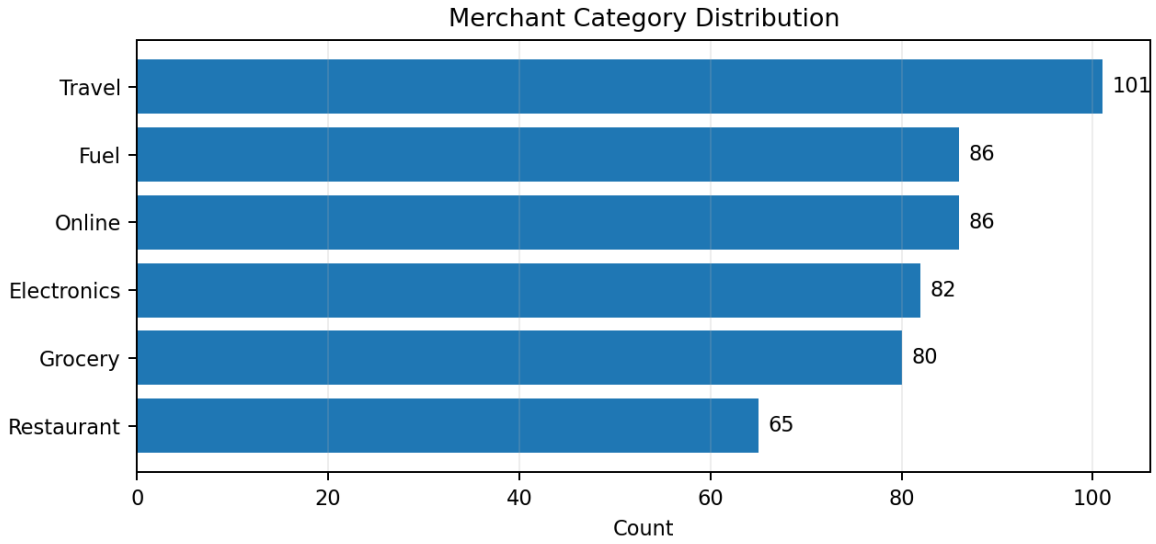


Transfer: 172 transactions (34.4%).

Debit: 168 transactions (33.6%).

Credit: 160 transactions (32.0%).

4.4 Merchant Category Distribution



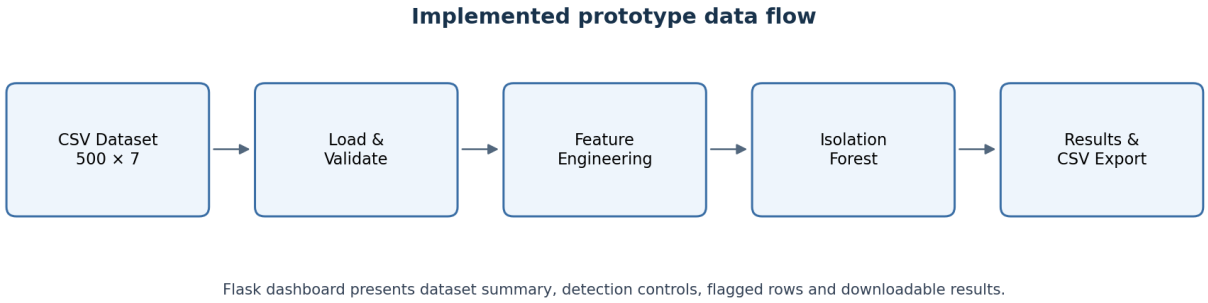
The largest merchant category is Travel. The category distribution is reasonably spread across six groups, but these counts describe only the supplied sample and should not be assumed to represent a real bank's portfolio.

4.5 Existing Label Distribution

The is_anomaly column contains 497 zeros and 3 ones. The positive rate is 0.6%. Because the label provenance is unverified and there are only 3 positive examples, supervised performance metrics such as precision, recall, F1, and false-positive rate would be unstable or misleading without a validated reference set.

5. System Design and Architecture

The application is organised as a local Flask web application that calls the existing Python data and machine-learning modules. The flow below describes the verified baseline path; unimplemented deep-learning models and external databases are not shown.



5.1 Main Components

Component	Responsibility	Verification evidence
Dataset loader / validation	Read CSV and check expected structure and values	Existing tests in tests.test_data_validation
Feature engineering	Parse time and amount values, encode categories, handle invalid/non-finite data	src/features/build_features.py
Isolation Forest baseline	Fit unsupervised detector and return scores/flags	src/models/baseline_iforest.py
Flask dashboard	Show overview and accept contamination input; present results	dashboard/app.py and dashboard templates
Results export	Write model outputs to a separate results CSV and serve download	reports/dashboard_results.csv after a run
EDA script	Generate visualisations and EDA findings	scripts/eda_phase2.py and reports/figures/

5.2 Data Flow

15. The application reads the original dataset.csv.
16. The loader and validation logic checks the input before feature preparation.
17. Feature engineering builds a numeric model matrix from permitted fields.
18. Isolation Forest estimates anomaly scores and predicted flags using the configured contamination value.
19. Predictions are appended to a separate results copy while preserving the original transaction fields and row order.
20. The dashboard renders results and provides a downloadable CSV without overwriting the source dataset.

6. Methodology and Feature Engineering

6.1 Why Unsupervised Detection

The supplied labels are sparse and their provenance is unknown. An unsupervised method is therefore used as an exploratory baseline rather than training a supervised classifier against potentially unreliable labels.

6.2 Isolation Forest

Isolation Forest isolates observations through random feature selection and split points. Observations that are isolated in fewer splits tend to be treated as more anomalous. The implementation wraps Scikit-learn's Isolation Forest and uses a fixed random_state for repeatable results under the same environment and inputs.

6.3 Feature Engineering

- Transaction amount is converted to numeric form and included as a continuous feature.
- Timestamp is parsed to derive time-based features; the raw timestamp is not passed directly to the model.
- Transaction type and merchant category are encoded as categorical features.
- account_id is excluded to avoid the model learning identifier-specific patterns.
- is_anomaly and anomaly_score are excluded from model input to avoid leakage from unverified labels or scores.

The prior implementation review reported 14 model features. The exact feature list is determined by the categories present in the input and the current feature-engineering code.

6.4 Contamination Parameter

Contamination controls the expected proportion of outliers used to define the decision threshold. It is a model setting, not an estimate validated against ground truth. The dashboard defaults to 0.01 and validates user input to the supported range used by the project. A setting of 0.01 does not mean that 1% of records are confirmed fraud.

7. Application Implementation

7.1 Technology Stack

Layer	Technology	Status
Programming language	Python 3.12	Used
Data handling	Pandas, NumPy	Used
Machine learning	Scikit-learn / Isolation Forest	Used
Web backend	Flask	Used
Frontend	HTML templates and CSS	Used
Visualisation	Matplotlib-generated EDA charts; dashboard bar displays	Used
TensorFlow / PyTorch / Autoencoder / LSTM-AE	Listed in brief as possible approaches	Not implemented in current prototype
Database / MLflow / PyOD / Plotly	Listed or possible tools in brief	Not verified as used in current prototype

7.2 Dashboard Features

- Dataset overview with row/column counts, transaction type counts, and merchant category counts.
- Important notice describing label provenance and evaluation limitations.
- Contamination input for running the Isolation Forest baseline.
- Results view with original transaction fields, model score, and predicted flag.
- CSV download of model results.
- Error handling for invalid contamination and unavailable results.

7.3 Running the Application

From the project directory, start the local Flask development server with:

python -m flask --app dashboard.app run --host 127.0.0.1 --port 5000

Then open http://127.0.0.1:5000 in the same computer's browser. The development server is for local testing and is not a production deployment.

8. Testing, Evaluation and Results

8.1 Automated Tests

The recorded verification command was `python -m unittest tests.test_data_validation tests.test_phase2 tests.test_phase2_review`. The project log reported 24 tests run and OK. The tests cover data validation, feature engineering, model baseline behavior, and review hardening.

8.2 Web Workflow Checks

Check	Reported result	Interpretation
GET / (homepage)	200 OK	Page responded successfully in the test client
POST /run (contamination 0.01)	200 OK	Detection route returned a successful response
GET /download after detection	200 OK	Download route served a results CSV
Invalid contamination input	Handled with error page	Validation path was exercised

These are application-level endpoint checks reported by the local test run. They do not prove performance on a production server or independent correctness of the anomaly labels.

8.3 Exploratory Model Output

The earlier baseline check reported 5 flagged transactions when contamination was set to 0.01. This is an exploratory model output, not a validated detection rate. The number of flagged records can change when the contamination value changes.

8.4 Performance Metrics

A reliable FPR, precision, recall, F1 score, or accuracy cannot be asserted from the current evidence because only three records have is_anomaly=1 and the source/meaning of those labels is not verified. The project brief asks for FPR below 0.1%; this target has NOT been demonstrated. No synthetic labels were added to make the metrics appear stronger.

8.5 Runtime

The recorded test suite completed in approximately 2.6 seconds in the reported environment. This is the test-suite duration, not a benchmark of the dashboard's model inference latency under load.

9. Ethical, Privacy and Operational Considerations

Anomaly flags in a financial context can affect customer experience and access to services. The prototype should be used only to prioritise human review, not to automatically block accounts or reject transactions.

9.1 False Positives and Appeals

The official brief sets a target false-positive rate below 0.1% and requires a clear appeal mechanism for flagged accounts. The current model has not demonstrated this FPR target, and no in-application appeal workflow is implemented. For any real deployment, the process should include human review, clear reasons for a flag where available, a way for an affected customer to contest the outcome, review by an authorised team, and audit records of the decision.

9.2 Privacy and Data Minimisation

The account_id field appears to be anonymised in the supplied dataset but is still an identifier-like field. It is excluded from model features. The project does not collect additional external data. Before submission or deployment, the dataset licence/source should be confirmed and the data dictionary should describe the anonymisation and permitted use based on evidence from the dataset provider.

9.3 AI Tools and Disclosure

AI assistance was used during development/documentation support. The learner should disclose the actual tools used and the extent of their contribution in the required appendix, following programme rules. This report does not claim that the work was completed without AI assistance.

9.4 DPDP Act 2023

Applicability and compliance with the Digital Personal Data Protection Act, 2023 must be assessed based on the actual data source, processing purpose, roles, consent/legal basis, and institutional guidance. This report does not certify legal compliance. The learner must confirm the dataset licence and applicable privacy obligations before signing the ethical declaration.

10. Limitations and Future Work

10.1 Current Limitations

- The dataset contains only 500 records and 3 positive labels; it is too small and weakly labelled for reliable FPR validation.
- The provenance and meaning of anomaly_score and is_anomaly are not verified.
- Only Isolation Forest is implemented; Autoencoder and LSTM-AE are not implemented.
- The false-positive-rate target below 0.1% has not been demonstrated.
- A formal appeal mechanism is not implemented in the software.
- The dashboard uses Flask's development server and is intended for local demonstration only.
- The current project does not establish production-level security, availability, monitoring, or auditability.

10.2 Future Enhancements

- Obtain a larger, source-documented dataset with independently validated anomaly labels.
- Evaluate precision, recall, F1, false-positive rate, and alert volume on a held-out validated set; report confidence intervals where appropriate.
- Compare Isolation Forest with an Autoencoder and LSTM-AE only if data volume and temporal structure justify the comparison.
- Add a human-review and appeal workflow before any consequential action.
- Add logging, access control, secure deployment, model versioning, monitoring, and drift checks for any production pilot.
- Update data_dictionary.csv so all seven columns have meaningful, verified descriptions.

11. Conclusion

Project 58 demonstrates an end-to-end unsupervised anomaly-detection prototype for financial transaction data. The verified software includes CSV loading and validation, feature engineering, an Isolation Forest baseline, a Flask dashboard, result display, and CSV export. The reported automated test suite passed 24 tests, and the homepage, detection, invalid-input handling, and download endpoints were checked.

The most important outcome is a working prototype with transparent limitations, not a claim of production-ready fraud detection. The supplied dataset has 500 records and only three positive labels of unverified provenance, so performance metrics—especially the brief's FPR target below 0.1%—remain unproven. Before final submission, the learner should complete personal fields, confirm the source licence, ensure the data dictionary descriptions are complete, prepare the demo, and sign the ethical declaration only after verifying each statement.

Submission Readiness Checklist

Deliverable	Current status from available evidence
Working prototype/application	Implemented; local workflow verified in project logs
Project report (minimum 8 pages)	This report draft prepared; learner should review before submission
EDA script with 3+ visualisations	Implemented; four EDA figures reported
Cleaned/preprocessed dataset.csv	File exists and has no missing values/duplicates in inspected copy; source/licence still to confirm
data_dictionary.csv all columns described	Needs final check/update if placeholder descriptions remain
Demo video (1–2 min) or live demo	Not verified as completed
Signed Ethical Compliance Declaration	Pending learner verification and signature

References

1. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. Proceedings of the Eighth IEEE International Conference on Data Mining, 413–422. https://doi.org/10.1109/ICDM.2008.17

2. Scikit-learn developers. IsolationForest API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

3. Pandas development team. pandas.read_csv documentation. https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

4. Pallets Projects. Flask Quickstart documentation. https://flask.palletsprojects.com/en/stable/quickstart/

5. HCLTech Career Shaper Programme. IV Year Industry Project — Project Brief, Project 58: Time-Series Anomaly Detection in Financial Transactions, Academic Year 2026–27 (provided source document).

Note: The official brief asks the mentor to validate and add programme references. Confirm the HCLTech programme guidelines reference with the programme coordinator before final submission.

Appendix A: Software Verification Summary

Item	Verified/Reported evidence
Test command	python -m unittest tests.test_data_validation tests.test_phase2 tests.test_phase2_review
Test outcome	24 tests reported passing (OK)
Homepage /	200 OK in test client
Detection /run	200 OK in test client
Download /download	200 OK after detection
Invalid contamination	Error page shown rather than unhandled failure
Leakage controls	is_anomaly, anomaly_score, account_id and raw timestamp excluded from model inputs
Original dataset integrity	Project log reported dataset.csv SHA-256 unchanged; uploaded report copy hash recorded in Section 3
Models	Isolation Forest implemented; Autoencoder and LSTM-AE not implemented
Deployment	Local Flask development server only

Appendix B: Items to Complete Before Submission

- Roll number: 2300320130234. Verify the mentor name and final submission date before submission.
- Review the report and ensure every statement matches the final local project.
- Update data_dictionary.csv with meaningful descriptions for all seven columns if placeholders remain.
- Confirm dataset source licence and data provenance.
- Capture genuine screenshots from the final running application and prepare a 1–2 minute demo or live demo.
- Complete the appeal-process description for the report and discuss the implementation gap with the mentor.
- Do not state FPR < 0.1% as achieved unless validated measurements support it.
- Review the ethical declaration and sign it only if each statement is true and supported.

Appendix C: Detailed Functional Requirements

This appendix expands the functional scope of the implemented prototype into verifiable requirements. It distinguishes features visible in the current local application from proposed enhancements that require additional implementation.

The prototype begins with the supplied CSV file and performs a sequence of data validation, feature preparation, anomaly scoring, result presentation, and export. Its purpose is to support an analyst's review, not to make an automatic financial decision.

ID	Requirement	Status / evidence
FR-01	Load transaction records from CSV	Implemented in data-loading module
FR-02	Validate required columns and data quality	Implemented; automated tests reported
FR-03	Summarise row/column counts and categories	Implemented in dashboard/EDA
FR-04	Derive model-ready numeric features	Implemented in feature engineering
FR-05	Run Isolation Forest with configurable contamination	Implemented
FR-06	Show prediction results in the web UI	Implemented; endpoint checks reported
FR-07	Download results as CSV	Implemented; endpoint check reported
FR-08	Provide a formal appeal workflow	Not implemented; future work
FR-09	Validate FPR below 0.1% on trusted labels	Not demonstrated
FR-10	Compare Autoencoder and LSTM-AE	Not implemented

Appendix D: Non-Functional Requirements

Non-functional requirements describe the quality attributes expected of the application. The current work is a student prototype, so these requirements are used as a review framework rather than as a claim that production standards have already been met.

Quality area	Prototype position	Recommended acceptance criterion
Reliability	Core paths tested locally	Repeated runs produce consistent output for the same data and settings
Performance	No formal latency benchmark	Measure load, feature-build, inference, and download time separately
Security	Local development server	Add authentication, secure configuration, and deployment hardening before pilot use
Privacy	No additional collection reported	Document data provenance, retention, access, and deletion
Usability	Dashboard supports a basic workflow	Test with representative reviewers and document task completion
Maintainability	Code split into modules	Document interfaces and keep tests for each module
Auditability	CSV results available	Record model version, settings, timestamp, and reviewer decision
Accessibility	Not formally evaluated	Check keyboard access, contrast, labels, and responsive layout

Appendix E: Data Dictionary and Field-Level Interpretation

The supplied dataset has seven fields. The table below describes their observed role in the current workflow. Where the source meaning is not independently documented, the interpretation is intentionally cautious.

Column	Observed role	Model input?	Caution
timestamp	Transaction date/time	Derived time features only	Timezone and source system are undocumented
account_id	Account-like identifier	No	Treat as identifier-like data; do not infer identity
trans_amount	Transaction amount	Yes	Currency/unit and transaction sign conventions should be confirmed
transaction_type	Transfer/Debit/Credit category	Encoded	Definitions may vary by source
merchant_category	Merchant group	Encoded	Category taxonomy is not documented
anomaly_score	Existing score field	No	Provenance and generating method are unknown
is_anomaly	Existing binary label	No	Only 3 positive records; label provenance unknown

The separate data_dictionary.csv file should be checked before submission. If it still contains placeholder descriptions, update it in the project folder. The report's explanations do not automatically modify that CSV file.

Appendix F: Data Quality Assessment Procedure

Data quality checks are important because model output is only as dependable as the data supplied to it. This project's validation phase checks the expected schema and basic integrity properties before the model is run.

The reviewed copy contained 500 rows and seven columns, no missing values, and no exact duplicate rows. Timestamps were sorted ascending from 1 January 2024 to 6 January 2024. These observations apply only to the inspected file, not to any future replacement dataset.

- Confirm that the expected column names exist and that required fields are not silently renamed.
- Parse timestamps with explicit error handling and inspect invalid or ambiguous values.
- Convert transaction amounts to numeric form and report any conversion failures.
- Check exact duplicate rows separately from possible duplicate transactions.
- Inspect categorical values for unexpected spellings, blanks, and category drift.
- Verify the origin and meaning of labels before computing supervised metrics.
- Preserve the source dataset and create derived outputs separately to support reproducibility.

Appendix G: Exploratory Data Analysis — Amounts

The observed transaction amount has a minimum of 6.07, a maximum of 1,471.92, a mean of approximately 152.04, and a median of 92.62. The mean is higher than the median, which is consistent with a right-skewed distribution in which a smaller number of large values pull the average upward.

An amount distribution is a descriptive view, not an anomaly detector by itself. Large amounts can be legitimate and small amounts can be suspicious in context. Interpretation should consider account behaviour, merchant type, transaction frequency, time, and verified business rules.

Before using amount-based thresholds operationally, the organisation would need to establish the currency, customer segments, transaction types, and relevant business context. Those details are not established by the current dataset alone.

Note: The report does not label any specific transaction as fraud; it describes statistical patterns in the supplied sample.

Appendix H: Exploratory Data Analysis — Time Patterns

The dataset spans approximately five days and nine hours, from 2024-01-01 00:26 to 2024-01-06 09:10. This interval is short for learning robust daily, weekly, seasonal, or holiday patterns. Time-derived features are useful for demonstrating the pipeline, but any seasonality conclusions should remain tentative.

A time-series anomaly system normally considers both individual transaction attributes and the sequence in which events occur. A transaction may be unusual because it occurs at an unexpected hour, follows a sudden change in activity, or deviates from a baseline for an account or merchant. The current implementation uses engineered timestamp features rather than a verified sequential deep-learning model.

- Check for missing time intervals and confirm timestamp timezone semantics.
- Aggregate volume at suitable intervals only after examining event frequency.
- Avoid random train/test splits if temporal leakage is possible; use chronological validation for future experiments.
- Compare patterns across multiple weeks or months before describing recurring seasonality.

Appendix I: Categorical Feature Analysis

The transaction type counts are Transfer 172, Debit 168, and Credit 160. These counts are relatively close in this sample and do not establish that the classes are balanced in a broader population.

The six merchant categories are Travel (101), Online (86), Fuel (86), Electronics (82), Grocery (80), and Restaurant (65). The most common category is Travel and the least common is Restaurant. Category frequency alone does not establish risk.

The implementation encodes categorical variables into numeric features for the Isolation Forest model. One-hot encoding prevents an arbitrary numeric ordering from being assigned to nominal categories. New categories at inference time should be handled explicitly to avoid schema mismatches.

Field	Observed values/counts	Interpretation
transaction_type	Transfer 172; Debit 168; Credit 160	Sample frequencies only
merchant_category	Travel 101; Online 86; Fuel 86; Electronics 82; Grocery 80; Restaurant 65	Six observed categories

Appendix J: Model Selection Rationale

Isolation Forest was selected as the current baseline because it is an unsupervised method that can be applied when reliable anomaly labels are scarce. It isolates observations using randomly selected features and split points; observations that are isolated more readily are treated as more anomalous.

The method is computationally practical for a small tabular dataset and is available in Scikit-learn. It provides a useful baseline for the project brief, but it does not automatically understand financial risk, customer intent, or temporal causality.

An Autoencoder or LSTM Autoencoder may be explored later if the dataset becomes sufficiently large and the sequence structure is meaningful. Such models should not be described as implemented in the current prototype.

- Strength: supports an unsupervised baseline without fitting to the unverified is_anomaly field.
- Strength: configurable contamination supports experimentation with alert volume.
- Limitation: anomaly scores are relative model outputs, not fraud probabilities.
- Limitation: results can be sensitive to feature construction and the selected contamination value.
- Limitation: evaluation requires reliable labels or a carefully designed expert-review protocol.

Appendix K: Feature Engineering Specification

Feature engineering transforms raw transaction fields into numeric inputs. In this project, transaction amount is used as a numeric feature; timestamp is parsed into time-derived values; transaction type and merchant category are encoded; account_id, raw timestamp, anomaly_score, and is_anomaly are excluded as direct inputs.

Excluding is_anomaly and anomaly_score helps avoid leakage from labels or prior scores whose provenance is not verified. Excluding account_id prevents the baseline from relying on identifier-specific patterns. These choices do not guarantee the absence of every form of leakage; the full transformation process should be reviewed when the data source is documented.

Source field	Transformation	Reason
trans_amount	Numeric conversion	Represent transaction magnitude
timestamp	Parse and derive time features	Represent time context without feeding raw strings
transaction_type	Categorical encoding	Represent nominal transaction types
merchant_category	Categorical encoding	Represent merchant grouping
account_id	Excluded	Identifier-like field
anomaly_score	Excluded	Unknown provenance
is_anomaly	Excluded	Avoid label leakage

Appendix L: Isolation Forest Configuration and Reproducibility

The model exposes a contamination setting. This parameter influences the expected fraction of outliers used to establish the decision threshold; it should not be confused with a measured prevalence of real fraud. When ground truth is not reliable, changing contamination primarily changes the number of records surfaced for review.

The reviewed implementation uses a configurable Isolation Forest baseline and a reproducible random state as documented in the implementation review. For a repeatable experiment, the input file hash, package versions, model parameters, and feature-generation version should be recorded alongside each result.

- Record the exact dataset version and cryptographic hash.
- Record the contamination value and all model parameters.
- Record the code revision and Python/package versions.
- Keep source data unchanged and export predictions to a separate file.
- Repeat the run to confirm stable outputs under fixed settings.

Appendix M: Evaluation Metrics and Their Meaning

A classifier-style evaluation requires trustworthy reference labels. If positive labels are unreliable, the computed confusion matrix may measure agreement with flawed labels rather than real-world anomaly detection. This is a major limitation in the current project.

Metric	Meaning	Evidence required
Precision	Share of flagged records that are truly anomalous	Verified positive/negative labels or expert adjudication
Recall	Share of true anomalies detected	Reliable positive labels with adequate sample size
F1 score	Harmonic mean of precision and recall	Reliable precision and recall estimates
False-positive rate	False alerts among true normal records	Reliable negative labels and a defined threshold
Alert volume	Number/share of records flagged	Can be measured from predictions, but is not accuracy
Inference latency	Time required to score records	Repeated timing under a documented environment

For the exploratory comparison at contamination 0.006, the prior implementation notes reported TP=0, FP=3, TN=494, FN=3 and an estimated FPR of about 0.604% against the supplied labels. Because label provenance is unknown, this is not a validated operational metric and does not meet the brief's <0.1% target.

Appendix N: Test Strategy

Testing is organised around data validation, feature generation, model execution, output handling, and the web workflow. The reported Phase 2 test run passed 24 tests. This confirms the tested code paths in that run; it does not prove that all edge cases, deployment conditions, or business outcomes are correct.

Test area	Example check	Expected outcome
Schema validation	Required columns missing	Clear validation failure
Numeric conversion	Invalid amount value	Handled error or documented rejection
Timestamp parsing	Malformed timestamp	Validation failure or controlled handling
Feature engineering	Valid mixed categorical input	Numeric feature matrix produced
Model configuration	Valid contamination value	Model runs
Model configuration	Invalid contamination value	Error handled without crashing the workflow
Dashboard	GET /	HTTP 200 for home page
Detection	POST /run	Results displayed for valid request
Download	GET /download after detection	CSV returned

Appendix O: Test Results and Interpretation

The recorded automated test command was `python -m unittest tests.test_data_validation tests.test_phase2 tests.test_phase2_review`. The project notes reported 24 tests passing in approximately 2.6 seconds.

The Flask endpoint checks reported a successful home page response, a successful detection request at contamination 0.01, and a successful download request after detection. Invalid contamination was handled by an error page. These checks should be repeated on the final submission copy before demonstration.

A test suite result is a software verification result. It must not be presented as model accuracy, fraud-detection success, production uptime, or proof that the false-positive-rate target has been achieved.

- Run the full test command after any code changes.
- Capture the final console output as evidence for the project review.
- Test the dashboard in a clean browser session.
- Verify downloaded CSV columns and row count.
- Confirm the original dataset is not overwritten by the model run.

Appendix P: Dashboard User Guide

The application is intended to run locally on the developer's computer. The Flask development server is suitable for demonstration and testing, not public production use.

- Open PowerShell and change directory to the project root.
- Run: python -m flask --app dashboard.app run --host 127.0.0.1 --port 5000
- Open http://127.0.0.1:5000 in a browser on the same computer.
- Review the dataset overview and the notice describing the unverified label provenance.
- Choose a valid contamination value and start anomaly detection.
- Inspect the predicted flags and scores as review recommendations only.
- Download the result CSV and retain it separately from the source dataset.
- Stop the local server with Ctrl+C after the demonstration.

Appendix Q: Developer Setup and Project Structure

The project is organised into data, feature, model, evaluation, dashboard, test, script, and report components. This separation makes it easier to test data handling independently from model execution and the user interface.

Path	Purpose
src/data/load.py	Load input data
src/data/validate.py	Validate schema and basic data integrity
src/features/build_features.py	Create numeric model features
src/models/baseline_iforest.py	Isolation Forest baseline
src/evaluation/evaluate.py	Evaluation helpers
scripts/eda_phase2.py	EDA workflow and charts
dashboard/	Local Flask application
tests/	Automated verification
reports/	Findings and generated figures
dataset.csv	Supplied transaction data
data_dictionary.csv	Column descriptions; verify for placeholders

The project should be run in the environment where its dependencies are installed. If preparing a ZIP, exclude virtual environments, cache folders, private credentials, and any .env file containing secrets.

Appendix R: Privacy and Data Minimisation Review

Financial data can be sensitive even when direct personal identifiers are absent. The account_id field is identifier-like and should be treated carefully. The project does not need to collect additional personal information to demonstrate anomaly detection.

Before any real-world pilot, the data owner should document the dataset source, legal basis or permissions for processing, retention period, access controls, deletion process, and whether the data is anonymised or pseudonymised. The student prototype alone cannot certify compliance with a law or organisational policy.

- Use the minimum fields required for the task.
- Avoid including names, account numbers, or direct identifiers in screenshots and demo videos.
- Restrict access to raw data and exported results.
- Do not upload real customer records to public repositories.
- Document the licence and permission for the supplied dataset.
- Obtain mentor or data-owner review before claiming compliance.

Appendix S: Responsible Use and Human Review

An anomaly detector produces a prioritisation signal, not a conclusion about intent or wrongdoing. An unusual transaction may have a legitimate explanation, and a normal-looking transaction may still be problematic. The user interface and report should communicate this distinction clearly.

The system should not automatically freeze accounts, deny services, or accuse a customer based only on the model score. Any consequential decision would require independent evidence, an authorised reviewer, a documented process, and a way for affected users to challenge incorrect outcomes.

- Show that flags are model-generated suggestions.
- Provide reviewers with the transaction context available under policy.
- Record reviewer decisions and reasons where appropriate.
- Define escalation and appeal ownership before operational deployment.
- Monitor false alerts and investigate systematic disparities.

Appendix T: False-Positive Target and Threshold Tuning

The project brief specifies a target false-positive rate below 0.1%. The current dataset and labels do not provide adequate evidence to demonstrate that target. At such a low rate, a sufficiently large and representative set of known-normal transactions is needed to estimate false positives reliably.

Threshold selection should be performed on a validation set that is separate from the final evaluation set. The chosen threshold should be assessed alongside recall, alert volume, and the operational capacity for review. A low false-positive rate alone can be misleading if it is achieved by flagging very few records and missing most anomalies.

- Acquire independently verified labels or a documented expert-review process.
- Split data chronologically where temporal leakage is possible.
- Tune threshold on a validation set, then evaluate once on a held-out set.
- Report confidence intervals and the number of normal examples.
- Document the trade-off between missed anomalies and false alerts.

Appendix U: Proposed Appeal and Review Workflow

A formal appeal mechanism is required by the project brief but is not implemented in the current software. The following is a design proposal for future work, not a claim about current application functionality.

Step	Proposed action	Record
1. Flag	Model adds a record to a review queue	Model version, score, timestamp
2. Review	Authorised reviewer examines permitted context	Reviewer ID and review time
3. Outcome	Reviewer marks likely benign, needs escalation, or confirmed issue	Reason code and notes
4. Notify	If a consequential action is considered, follow approved policy	Communication record
5. Appeal	Affected person can request review through a defined channel	Case ID and submission date
6. Reassess	Independent reviewer considers new evidence	Decision and explanation
7. Monitor	Team audits outcomes and appeal reversals	Periodic metrics

Appendix V: Risk Register

The risk register lists material risks that should be reviewed before any use beyond the student demonstration. Ratings are qualitative and should be updated with the mentor and data owner.

Risk	Potential impact	Mitigation / status
Unverified labels	Misleading evaluation	High priority: document provenance; obtain verified labels
Small sample	Unstable thresholds	Use a larger representative dataset
False positives	Unnecessary investigation or customer friction	Human review; tune on verified data
False negatives	Missed suspicious activity	Evaluate recall on trusted labels
Data leakage	Inflated results	Exclude label/score fields and review pipeline
Privacy exposure	Disclosure of sensitive records	Data minimisation, access control, safe demo
Development server exposed	Security risk	Keep local only; harden before deployment
Unclear appeal process	Unfair or unreviewable outcomes	Implement formal review and appeal workflow

Appendix W: Deployment and Security Boundaries

The verified application runs with Flask's development server. This is a local demonstration setup and should not be exposed directly to the public internet. A production deployment would require a production WSGI server, reverse proxy, TLS, authentication, authorisation, secret management, dependency review, logging, monitoring, and a documented incident process.

- Keep the binding host at 127.0.0.1 for local demonstrations unless a controlled environment is explicitly configured.
- Do not commit credentials, API keys, or private environment files.
- Validate and limit uploaded files if user uploads are enabled.
- Use dependency pinning and vulnerability checks for a controlled deployment.
- Apply role-based access to sensitive transaction data.
- Log security-relevant events without exposing unnecessary transaction details.
- Test backups, recovery, and retention policies before operational use.

Appendix X: Monitoring and Model Maintenance Plan

Anomaly patterns can change as customer behaviour, merchant mix, transaction systems, and fraud strategies evolve. A model that performs acceptably on one sample may become less useful over time. The current prototype does not include a production monitoring system; the following is a proposed plan.

Monitoring item	Purpose	Suggested cadence
Alert rate	Detect sudden changes in number of flags	Each run / daily in a pilot
Feature drift	Detect input distribution changes	Weekly or monthly
Verified outcomes	Measure precision and missed anomalies	As labels become available
Threshold review	Confirm alert volume remains manageable	After each validated evaluation cycle
Data quality	Detect schema and missing-value changes	Every ingestion
Model version	Reproduce historical decisions	For every model run

Appendix Y: Project Plan and Milestones

The project brief defines a four-week individual project. The following milestone plan is a structured summary of a reasonable workflow for this type of project. It should be reconciled with the learner's actual work log before submission; it is not a substitute for dated evidence.

Phase	Activities	Evidence / deliverable
Week 1	Understand brief, inspect dataset, validate schema	Dataset profile and validation tests
Week 2	Perform EDA and build feature transformations	EDA script, charts, feature matrix
Week 3	Implement Isolation Forest and dashboard integration	Baseline model and local application
Week 4	Run tests, document results, prepare report/demo	Test output, report, demo, checklist

Appendix Z: Reproducibility Checklist

Reproducibility enables a reviewer to repeat the analysis and understand why a result was produced. The following items should be retained with the submission wherever feasible.

- Dataset file name, row/column count, and hash.
- Python version and dependency versions.
- Exact test command and console output.
- Model name, parameters, and random state.
- Feature list and encoding strategy.
- Run timestamp and output filename.
- Dashboard screenshot captured from the actual running application.
- Notes describing any differences between report and final code.
- No unverified metric should be copied into the final summary.

Appendix AA: Glossary of Terms

Term	Definition
Anomaly	An observation that differs from an expected pattern; not automatically fraud
Time series	Observations indexed in time order
Unsupervised learning	Learning patterns without using verified target labels
Isolation Forest	Tree-based method that isolates observations to identify outliers
Contamination	Model setting influencing the expected proportion of outliers
Feature engineering	Transforming raw fields into useful model inputs
Data leakage	Use of information that improperly reveals the target or future outcome
False positive	Normal observation incorrectly flagged as anomalous
False negative	Anomaly that the model fails to flag
Precision	Share of flagged items that are truly positive
Recall	Share of true positives that are flagged
Drift	Change in input patterns or relationship over time
Ground truth	Trusted reference label or outcome
Human-in-the-loop	A workflow in which a person reviews model outputs

Appendix AB: Sample Demonstration Script

The following is a suggested 1–2 minute demonstration sequence. Record the actual running application and avoid showing confidential data or claiming that the model identifies confirmed fraud.

- Introduce the project title and explain that it is an unsupervised anomaly-detection prototype.
- Show the dataset overview and state that the inspected dataset has 500 records and seven columns.
- Briefly show the EDA visualisations: amount distribution, transaction volume over time, transaction types, and merchant categories.
- Run the Isolation Forest baseline with a valid contamination setting.
- Explain that the model flags unusual records for human review; the score is not a fraud probability.
- Open the result table and download the CSV.
- Mention that labels are not verified and the <0.1% false-positive target has not been demonstrated.

Appendix AC: Submission Evidence Checklist

This checklist is intended to reduce the risk of submitting incomplete or unsupported claims. Complete each item only after verifying it in the final project folder.

- Final PDF report opens and page numbers render correctly.
- Editable DOCX report opens and has no blank or broken pages.
- Roll number 2300320130234 is correct throughout the report.
- Academic year and programme wording match the instructions provided by the coordinator.
- dataset.csv is the same dataset used for the reported results.
- data_dictionary.csv has meaningful descriptions for every field.
- EDA script and at least three visualisations are included.
- Source code and tests are included in a clean archive if the portal accepts it.
- Demo video or live demonstration is ready.
- Ethical declaration is completed only after verification and signed by the learner.
- Mentor name and final submission date are filled where required.

Appendix AD: Final Technical Status and Open Items

The project currently demonstrates a local, end-to-end baseline pipeline: CSV input, validation, feature engineering, Isolation Forest scoring, dashboard presentation, and CSV export. The recorded automated test run reported 24 passing tests.

The inspected dataset has 500 records, seven columns, and three records labelled as anomalies. The provenance of the existing labels and anomaly scores is not known. Therefore, any metrics calculated against those labels are exploratory and should not be presented as verified real-world performance.

The project brief's false-positive-rate target of less than 0.1% remains unverified. Autoencoder and LSTM-AE comparison, a formal appeal mechanism, production deployment, and production security controls remain future work.

- Confirm source licence and dataset provenance.
- Correct placeholder descriptions in data_dictionary.csv if still present.
- Add a real application screenshot and demo evidence.
- Confirm programme-specific formatting requirements with the mentor.
- Sign the ethical declaration only after verifying each statement.